CMake Mastery

From Single Binary to Real-World Build Graphs

Python/C++ 개발자를 위한 실전 빌드 시스템 책

단일 실행 파일, 라이브러리 분리, 외부 패키지, 테스트, 설치,
Presets, Toolchain, LLVM/MLIR 연동까지 한 흐름으로 정리한 학습서

2026

Contents

1 왜 CMake를 마스터해야 하는가 **1**
1.1 CMake가 실제로 하는 일 . . . 1

2 가장 작은 CMake 프로젝트 **3**
2.1 최소 구성 . . . 3
2.2 자주 쓰는 기본 설정 . . . 3
2.3 빌드 디렉토리를 분리하는 이유 . . . 4

3 예약어, 명령, 옵션을 어떻게 읽어야 하는가 **5**
3.1 먼저 정리: CMake에 진짜 "예약어"가 있는가 . . . 5
3.2 현재 예제 코드 한 줄씩 해석 . . . 5
3.3 자주 보는 명령을 상세히 이해하기 . . . 6
3.4 옵션을 읽을 때 흔히 놓치는 포인트 . . . 9
3.5 무엇을 어디까지 외워야 하는가 . . . 9
3.6 현재 당신 수준에서 특히 기억해야 할 것 . . . 12

4 타겟 중심 사고 **15**
4.1 파일이 아니라 타겟을 본다 . . . 15
4.2 PUBLIC, PRIVATE, INTERFACE . . . 15
4.3 header-only 라이브러리 . . . 16

5 프로젝트 구조를 어떻게 쪼개는가 **17**
5.1 권장 디렉토리 구조 . . . 17
5.2 루트와 서브디렉토리 책임 분리 . . . 17
5.3 언제 add_subdirectory를 쓰나 . . . 18

6 컴파일 옵션과 정의를 다루는 법 **19**
6.1 전역 옵션보다 타겟 옵션 . . . 19
6.2 빌드 타입과 generator expression . . . 19
6.3 경고 플래그 모듈화 . . . 20

7 링크, 라이브러리, 그리고 왜 자꾸 undefined reference가 나는가 **21**
7.1 컴파일과 링크는 다른 단계다 . . . 21
7.2 정적 라이브러리와 공유 라이브러리 . . . 21
7.3 의존성 전파 예시 . . . 21

8 외부 패키지를 찾고 연결하기 **23**
8.1 find_package의 두 세계 . . . 23
8.2 Imported target을 써야 하는 이유 . . . 23
8.3 패키지를 못 찾을 때 보는 것 . . . 23

9 테스트, install, package로 이어지는 흐름 **25**
9.1 CTest 연결 . . . 25
9.2 install 규칙 . . . 25
9.3 재사용 가능한 패키지로 내보내기 . . . 25

10 configure 단계에서 파일을 생성하는 법 **27**
10.1 configure_file . . . 27
10.2 file(GENERATE)와의 차이 . . . 27

11 Presets와 Toolchain으로 팀 개발 수준을 올리기 **29**
11.1 CMakePresets.json . . . 29
11.2 Toolchain 파일 . . . 29

12 LLVM/MLIR 프로젝트를 읽는 눈 **31**
12.1 왜 별도 장이 필요한가 . . . 31
12.2 핵심 패턴 . . . 31
12.3 라이브러리 계층을 읽는 법 . . . 32
12.4 빈출 문제 . . . 32

13 디버깅과 안티패턴 **33**
13.1 문제가 생겼을 때 순서 . . . 33
13.2 유용한 명령 . . . 33
13.3 피해야 할 안티패턴 . . . 33

14 당장 실전에 적용할 로드맵 **35**
14.1 1단계: 작은 CLI 프로젝트 . . . 35
14.2 2단계: 라이브러리 분리 . . . 35
14.3 3단계: 외부 패키지와 설치 . . . 35
14.4 4단계: 대형 생태계 읽기 . . . 35

15 Final Quiz **37**

A 암기보다 중요한 체크포인트 **39**

B 빠른 명령 치트시트 **41**

Chapter 1

왜 CMake를 마스터해야 하는가

당신의 목표는 단순히 `cmake -B build -S .`를 외우는 것이 아닙니다. **프로젝트 구조를 설계하고, 타겟 단위로 의존성을 관리하고, LLVM/MLIR 같은 큰 생태계와 연결해도 버티는 빌드 감각**을 갖추는 것이 핵심입니다.

이 책의 목표

1. 작은 C++ 프로젝트를 스스로 빌드할 수 있다.
2. 헤더 경로, 컴파일 옵션, 링크 의존성을 타겟 기준으로 설계할 수 있다.
3. 외부 패키지와 테스트, 설치, 재사용 가능한 패키지 구성을 이해한다.
4. LLVM/MLIR처럼 크고 복잡한 생태계에서 CMake 파일을 읽고 고칠 수 있다.

Python 개발자 관점

Python에서 익숙한 것은 `pip install`, `pyproject.toml`, 가상환경, 테스트 러너입니다. C++에서는 그 역할이 여러 층으로 나뉩니다. 컴파일러, 링커, 빌드 디렉토리, 패키지 설정, 설치 경로를 명시적으로 다뤄야 합니다. CMake는 이 복잡성을 정리해 주는 메타 빌드 시스템입니다.

1.1 CMake가 실제로 하는 일

CMake는 컴파일러가 아닙니다. `ninja`, `make`, Xcode, Visual Studio 같은 **실제 빌드 도구용 설정 파일**을 생성합니다. 즉, CMake의 핵심 일은 "프로젝트의 빌드 그래프를 선언"하는 것입니다.

기본 흐름

```
cmake -S . -B build          # configure: 빌드 그래프 생성
cmake --build build          # build: 실제 컴파일링크/
ctest --test-dir build       # test: 테스트 실행
cmake --install build        # install: 산출물 배치
```

배운 것 정리

- configure 단계와 build 단계는 다르다.
- CMake를 읽을 때는 "무슨 파일을 컴파일하나"보다 "무슨 타겟을 정의하나"를 먼저 봐야 한다.
- 프로젝트가 커질수록 파일보다 **타겟 간 관계**를 이해하는 것이 더 중요하다.

Quiz

Q1. CMake가 직접 하는 일과 하지 않는 일을 구분해 보자.

1. `g++` 실행 자체
2. 타겟 사이 의존성 그래프 모델링
3. Ninja 파일 생성
4. 소스 코드를 최적화해서 더 빠르게 만드는 것

정답: 2, 3만 CMake의 역할. 1은 생성된 빌드 시스템이 수행하고, 4는 컴파일러/링커의 역할이다.

Chapter 2

가장 작은 CMake 프로젝트

2.1 최소 구성

CMakeLists.txt 최소 버전

```
cmake_minimum_required(VERSION 3.20)
project(kv_cli LANGUAGES CXX)

add_executable(kv
  src/main.cpp
)
```

src/main.cpp

```
#include <iostream>

int main() {
  std::cout << "kv-cli" << std::endl;
  return 0;
}
```

2.2 자주 쓰는 기본 설정

```
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
```

의미

- `CMAKE_CXX_STANDARD 20`: C++20 사용 의도
- `REQUIRED ON`: 지원 안 하면 에러

- `EXTENSIONS OFF`: `gnu++20` 대신 `c++20` 선호

주의

실전에서는 전역 변수보다 `target_compile_features()`가 더 안전하다. 전역 설정은 서브프로젝트와 섞일 때 충돌이 생기기 쉽다.

2.3 빌드 디렉토리를 분리하는 이유

```
cmake -S . -B build/debug -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake -S . -B build/release -G Ninja -DCMAKE_BUILD_TYPE=Release
```

소스 디렉토리와 빌드 디렉토리를 분리하면, 설정 조합을 여러 개 유지할 수 있고 산출물이 저장소를 오염시키지 않습니다.

Quiz

Q2. 아래 중 권장되는 흐름은?

A. 프로젝트 루트에서 `make`

B. 소스 루트에 `CMakeCache.txt`와 객체 파일을 섞어서 생성

C. 별도 `build/` 디렉토리에서 configure / build / test / install 수행

정답: C

Chapter 3

예약어, 명령, 옵션을 어떻게 읽어야 하는가

3.1 먼저 정리: CMake에 진짜 "예약어"가 있는가

엄밀하게 말하면 CMake는 C++처럼 강한 의미의 예약어 목록이 있는 언어가 아닙니다. 실전에서 우리가 "예약어"라고 부르는 것은 보통 세 층입니다.

1. **명령(command)**: `project()`, `add_library()`, `find_package()`처럼 호출하는 것
2. **인자/키워드(keyword-like argument)**: `PUBLIC`, `PRIVATE`, `REQUIRED`, `CONFIG`, `VERSION`처럼 명령 안에서 의미를 바꾸는 것
3. **변수(variable)**: `CMAKE_CXX_STANDARD`, `PROJECT_SOURCE_DIR`, `BUILD_TESTING`처럼 상태를 들고 있는 것

그래서 CMake 파일을 읽을 때는 "이 단어가 문법 예약어인가?"보다 "이 단어가 명령인지, 옵션인지, 변수인지"를 먼저 분류해야 합니다.

CMake 읽기의 기본 질문

1. 지금 보는 줄이 **무슨 타겟을 만들거나 수정하는가**?
2. 이 줄은 **configure 시점**에 평가되나, build / generate 시점에 반영되나?
3. 이 설정은 **현재 타겟만** 바꾸나, **소비자에게 전파**되나?
4. 이 값은 **전역 변수**인가, **타겟 속성**인가?

3.2 현재 예제 코드 한 줄씩 해석

가장 작은 예제 다시 보기

```
cmake_minimum_required(VERSION 3.20)
project(kv_cli LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

add_executable(kv
  src/main.cpp
```

```
)
```

줄	의미
`cmake_minimum_required(VERSION 3.20)`	이 프로젝트는 최소 CMake 3.20 문법과 정책을 기준으로 작성되었다는 선언이다. 너무 낮은 버전의 CMake로 열면 바로 실패시킨다. 동시에 여러 CMake 정책(policy)의 기본 동작도 이 버전에 맞춰진다.
`project(kv_cli LANGUAGES CXX)`	프로젝트 이름을 `kv_cli`로 정하고, C++ 언어를 활성화한다. 이 줄이 지나야 C++ 컴파일러 탐색과 관련 변수 초기화가 제대로 이뤄진다.
`set(CMAKE_CXX_STANDARD 20)`	이후 생성되는 C++ 타겟의 기본 언어 표준을 C++20으로 요청한다. "반드시 이걸 써라"라기보다는 기본 의도를 지정하는 전역 변수에 가깝다.
`set(CMAKE_CXX_STANDARD_REQUIRED ON)`	컴파일러가 C++20을 제대로 지원하지 않으면 다른 하위 표준으로 조용히 내려가지 말고 실패하라는 뜻이다.
`set(CMAKE_CXX_EXTENSIONS OFF)`	`gnu++20` 같은 컴파일러 확장 모드보다 표준 모드 `c++20`를 선호한다. 이식성을 높이는 데 유리하다.
`add_executable(kv src/main.cpp)`	`kv`라는 실행 파일 타겟을 만들고, 그 소스로 `src/main.cpp`를 연결한다. 여기서 핵심은 파일 나열이 아니라 "kv라는 타겟이 존재하게 된다"는 점이다.

Python 개발자 관점

Python에서 `pyproject.toml`은 빌드 메타데이터와 패키지 설정을 담습니다. CMake의 각 줄은 그보다 더 낮은 레벨입니다. 컴파일러 활성화, 언어 표준, 실제 링크 타겟 생성 같은 단계가 명시적으로 드러납니다.

3.3 자주 보는 명령을 상세히 이해하기

`cmake_minimum_required(VERSION ...)`

- `VERSION`은 명령의 필수 키워드다.
- 단순 버전 체크 이상으로, CMake 정책의 기준선을 잡는 역할이 중요하다.
- 실전에서는 프로젝트가 실제로 테스트한 최소 버전을 적어야 한다. "대충 최신"을 적으면 오래된 환경에서 괜히 깨지고, 너무 낮게 적으면 실제로는 지원하지 않는 기능을 쓰면서 거짓말하게 된다.

주의

`cmake_minimum_required()`는 거의 항상 루트 `CMakeLists.txt` 최상단에 둔다. CMake 정책이 뒤늦게 바뀌면 문서 해석과 동작이 꼬일 수 있다.

`project(...)`

```
project(kv_cli VERSION 0.1.0 LANGUAGES CXX)
```

항목	의미
`kv_cli`	프로젝트 이름. `PROJECT_NAME`, `CMAKE_PROJECT_NAME` 같은 변수에 반영된다.
`VERSION 0.1.0`	프로젝트 버전. 설치 패키지, configure 파일, 문서 메타데이터에 재사용하기 좋다.
`LANGUAGES CXX`	C++만 활성화. 필요하면 `C`, `CUDA`, `OBJCXX` 등을 추가할 수 있다. 안 쓰는 언어를 켜지 않는 것이 configure 시간을 줄이고 의도를 선명하게 한다.

기억 포인트: `project()`는 단순 이름표가 아니라 **언어 활성화 트리거**라고 기억하면 된다.

`set(...)`

`set()`은 가장 남용되기 쉬운 명령입니다. 문제는 너무 범용이라서, 값을 넣는 순간 "지역 변수", "캐시 변수", "전역 관습 변수", "하위 디렉토리에 전파되는 설정"이 뒤섞이기 쉽다는 점입니다.

```
set(CMAKE_CXX_STANDARD 20)
set(KV_ENABLE_LOGGING ON)
set(MY_PATH "${CMAKE_CURRENT_SOURCE_DIR}/data")
```

- 첫 줄은 CMake가 특별히 읽는 **관습 변수**를 세팅하는 예다.
- 둘째 줄은 우리가 정의한 일반 변수다.
- 셋째 줄은 문자열 결합이다. `"${...}"`는 변수 치환.

주의

`set()`로 모든 문제를 해결하려는 습관은 좋지 않다. 사용자가 끄고 켤 옵션이면 `option()`, 타겟 컴파일 정의면 `target_compile_definitions()`, include path면 `target_include_directories()`가 더 적절하다.

`option(...)`

```
option(KV_BUILD_TESTS "Build tests" ON)
```

- 첫 번째 인자: 캐시 변수 이름
- 두 번째 인자: 사용자에게 보이는 설명
- 세 번째 인자: 기본값

이 명령은 "사용자가 configure 시점에 바꿀 수 있는 불리언 스위치"를 만든다. `cmake -DKV_BUILD_TESTS=OFF ...`처럼 덮어쓸 수 있다.

`add_executable(...)` / `add_library(...)`

이 둘은 CMake의 중심입니다. 프로젝트를 읽을 때 가장 먼저 찾아야 하는 줄입니다.

```
add_library(kv_core src/store.cpp)
add_executable(kv_cli src/main.cpp)
```

- 이 줄이 있어야 타겟이 생긴다.
- 이후의 `target_*` 명령은 거의 모두 이 타겟을 수정한다.
- 소스 목록은 "이 타겟을 구성하는 재료"일 뿐이다. 진짜 주인공은 타겟 이름이다.

`target_include_directories(...)`

```
target_include_directories(kv_core
  PUBLIC include
  PRIVATE src
)
```

옵션	의미
`PUBLIC`	현재 타겟도 쓰고, 이 타겟을 링크하는 소비자도 필요하다. 보통 공개 헤더 위치.
`PRIVATE`	현재 타겟의 구현에서만 필요하다. 소비자에게는 전달하지 않는다.
`INTERFACE`	현재 타겟 자체는 안 쓰고, 소비자에게만 전달한다. header-only 라이브러리나 usage requirement 묶음에 많다.

현재 코드의 의미: `kv_core`의 공개 헤더는 `include/` 아래에 있고, 내부 구현은 `src/`에 있는 비공개 헤더를 쓸 수도 있다는 뜻입니다.

`target_link_libraries(...)`

```
target_link_libraries(kv_cli PRIVATE kv_core)
```

이 한 줄의 의미는 세 겹입니다.

1. 링크 단계에서 `kv_core`를 `kv_cli`에 연결한다.
2. `kv_core`가 들고 있는 usage requirement를 `PRIVATE`/`PUBLIC` 규칙에 따라 전달받는다.
3. "이 실행 파일은 이 라이브러리에 의존한다"는 빌드 그래프 관계를 명시한다.

여기서 `PRIVATE`의 뜻: `kv_cli`를 누가 또 링크해 가는 상황은 거의 없으므로, 보통 실행 파일 링크는 `PRIVATE`라고 기억하면 된다.

`find_package(...)`

```
find_package(fmt REQUIRED CONFIG)
```

옵션	의미
`fmt`	찾고 싶은 패키지 이름
`REQUIRED`	못 찾으면 경고가 아니라 configure 실패
`CONFIG`	패키지가 제공하는 `fmtConfig.cmake` 계열 파일을 우선 찾겠다는 의도

현재 코드의 의미: "fmt가 설치되어 있다고 가정하고, 그 패키지가 제공하는 현대식 imported target 메타데이터를 읽어와라. 없으면 조용히 넘어가지 말고 바로 실패해라" 입니다.

install(...)

```
install(TARGETS kv_core kv_cli
  RUNTIME DESTINATION bin
  LIBRARY DESTINATION lib
  ARCHIVE DESTINATION lib
)
```

- `TARGETS`: 어떤 타겟을 설치할지
- `RUNTIME DESTINATION`: 실행 파일이 갈 위치
- `LIBRARY DESTINATION`: 공유 라이브러리가 갈 위치
- `ARCHIVE DESTINATION`: 정적 라이브러리, import library가 갈 위치

이건 단순 복사가 아니라 "이 프로젝트를 다른 곳에서 소비 가능한 산출물로 정리하라"는 선언입니다.

3.4 옵션을 읽을 때 흔히 놓치는 포인트

옵션은 명령마다 의미가 다르다

`VERSION`, `REQUIRED`, `CONFIG`, `PUBLIC`, `PRIVATE` 같은 단어는 영어 단어라서 감으로 읽기 쉽지만, **어느 명령 안에 들어갔는지**에 따라 정확한 의미가 달라집니다. 예를 들어 `VERSION`은 `cmake_minimum_required()`에서는 최소 버전 문법이고, `project()`에서는 프로젝트 버전 메타데이터입니다.

단어	주로 나오는 곳	실전 해석
`VERSION`	`cmake_minimum_required`, `project`	최소 도구 버전인지, 프로젝트 자체 버전인지 문맥으로 구분
`REQUIRED`	`find_package`	없으면 진행하지 말고 실패
`CONFIG`	`find_package`	Config mode를 우선 사용
`PUBLIC`	`target_*_*` 계열	현재 타겟과 소비자 모두에 전파
`PRIVATE`	`target_*_*` 계열	현재 타겟에만 적용
`INTERFACE`	`target_*_*` 계열	소비자에게만 적용

3.5 무엇을 어디까지 외워야 하는가

레벨 1: 바로 손으로 쓸 수 있어야 하는 것

- `cmake_minimum_required`
- `project`
- `add_executable`, `add_library`
- `target_link_libraries`
- `target_include_directories`
- `target_compile_features`

- `find_package`
- `option`
- `add_subdirectory`

이 레벨은 **검색 없이도 대충 뼈대를 칠 수 있어야** 합니다. 특히 `PUBLIC` / `PRIVATE` / `INTERFACE`의 감각은 암기가 아니라 반사적으로 나와야 합니다.

레벨 1 명령은 예제까지 같이 기억

아래 예제는 "지금 바로 빈 프로젝트에 손으로 칠 수 있어야 하는 최소 패턴"입니다. 완벽히 토씨까지 외우라는 뜻은 아니고, **명령이 하는 역할과 대표적인 쓰임**이 바로 떠올라야 한다는 뜻입니다.

cmake_minimum_required

```
cmake_minimum_required(VERSION 3.20)
```

의미: 이 프로젝트는 최소 CMake 3.20을 요구한다. 너무 낮은 버전이면 configure 자체를 막는다.

project

```
project(kv_cli VERSION 0.1.0 LANGUAGES CXX)
```

의미: 프로젝트 이름을 `kv_cli`로 두고, 버전은 `0.1.0`, 활성 언어는 C++만 사용한다.

add_executable

```
add_executable(kv_cli
  src/main.cpp
)
```

의미: `kv_cli`라는 실행 파일 타겟을 만들고, 진입점 소스로 `src/main.cpp`를 사용한다.

add_library

```
add_library(kv_core
  src/store.cpp
  src/command.cpp
)
```

의미: 재사용 가능한 `kv_core` 라이브러리 타겟을 만들고, 핵심 구현을 그 안으로 모은다.

target_link_libraries

```
target_link_libraries(kv_cli PRIVATE kv_core)
```

의미: `kv_cli`는 `kv_core`를 링크해서 사용한다. 실행 파일 기준으로는 거의 항상 `PRIVATE`로 시작하면 된다.

target_include_directories

```
target_include_directories(kv_core
  PUBLIC include
)
```

의미: `kv_core`의 공개 헤더는 `include/` 아래에 있고, 이 라이브러리를 쓰는 소비자도 그 경로를 받아야 한다.

target_compile_features

```
target_compile_features(kv_core PUBLIC cxx_std_20)
```

의미: `kv_core`는 C++20 기능을 요구하며, 이 라이브러리의 공개 인터페이스를 소비하는 쪽도 그 요구를 이어받을 수 있다.

find_package

```
find_package(fmt REQUIRED CONFIG)
```

의미: fmt 패키지를 Config mode로 찾고, 없으면 조용히 넘어가지 말고 configure를 실패시킨다.

option

```
option(KV_BUILD_TESTS "Build tests" ON)
```

의미: 사용자가 `-DKV_BUILD_TESTS=OFF`처럼 켜고 끌 수 있는 불리언 옵션을 만든다.

add_subdirectory

```
add_subdirectory(src)
add_subdirectory(tests)
```

의미: `src/CMakeLists.txt`, `tests/CMakeLists.txt`를 현재 빌드 그래프에 포함시켜 하위 타겟 정의를 읽어온다.

레벨 2: 이름은 기억하고 필요할 때 문서 보면 되는 것

- `install`
- `configure_file`
- `add_test`, `include(CTest)`

- `set_target_properties`
- generator expression 문법
- export/package 관련 세부 명령

이 레벨은 "무슨 명령이 있는지"와 "어떤 문제에 쓰는지"까지만 기억하면 충분합니다. 세부 옵션 순서까지 암기할 필요는 없습니다.

레벨 3: 검색해서 써도 되는 것

- 복잡한 `install(EXPORT ...)` 조합
- `file(GENERATE)`의 세부 시맨틱
- Toolchain 파일 변수 세트
- CPack, package registry, 고급 policy
- LLVM/MLIR 전용 매크로 이름과 세부 인자

이건 자주 안 쓰므로 외우기보다 **존재를 알고 문서에서 정확히 확인하는 것**이 낫습니다.

실전 체크리스트

- 암기 대상 1순위는 명령 이름보다 **타겟 중심 사고**다.
- 2순위는 `PUBLIC`/`PRIVATE`/`INTERFACE`다.
- 3순위는 configure/build/install/test의 단계 구분이다.
- 나머지는 "이 문제를 풀 때 무슨 명령을 찾아야 하는가"를 아는 정도면 충분하다.

3.6 현재 당신 수준에서 특히 기억해야 할 것

당신은 초보 입문자를 넘어서, 이미 C++ 프로젝트와 LLVM/MLIR 계열 CMake를 읽고 싶은 단계에 있습니다. 이 단계에서 중요한 건 문법 조각보다 다음 네 가지입니다.

1. **타겟을 만든다**: `add_library`, `add_executable`
2. **타겟을 수정한다**: `target_link_libraries`, `target_include_directories`, `target_compile_features`
3. **외부 세계를 끌어온다**: `find_package`, `add_subdirectory`
4. **사용자 제어와 산출물 배치를 만든다**: `option`, `install`, `add_test`

이 네 분류만 머릿속에 고정되어 있으면, 처음 보는 CMake 파일도 훨씬 빨리 읽힙니다.

Quiz

Q2-1. 아래 줄을 한국어로 해석해 보라.

```
target_link_libraries(kv_cli PRIVATE kv_core)
```

정답: `kv_cli` 실행 파일은 `kv_core` 라이브러리에 의존하며, 그 관계는 현재 타겟에만 적용된다.

Quiz

Q2-2. `find_package(fmt REQUIRED CONFIG)`에서 `REQUIRED`를 빼면 무엇이 달라지는가?
정답: fmt를 못 찾았을 때 즉시 실패하지 않고, 이후 로직에서 조건 분기나 별도 처리를 할 여지가 생긴다. 단 대부분의 실전 프로젝트에서는 정말 필요한 패키지라면 `REQUIRED`를 두는 편이 더 명확하다.

Quiz

Q2-3. 지금 당장 암기해야 할 우선순위를 쓰면?
정답: 타겟 생성 명령, 타겟 수정 명령, `PUBLIC`/`PRIVATE`/`INTERFACE`, `find_package`, configure/build/install/test 단계 구분.

Chapter 4

타겟 중심 사고

4.1 파일이 아니라 타겟을 본다

CMake 초보는 보통 "이 파일을 어떻게 추가하지?"를 먼저 생각합니다. 그러나 유지보수 가능한 CMake는 "이 타겟은 어떤 API를 노출하고, 무엇에 링크되고, 어떤 include path와 compile definition을 소비자에게 전파하는가?"로 설계합니다.

좋은 방향의 CMake

```
add_library(kv_core
  src/store.cpp
  src/command.cpp
)

target_include_directories(kv_core
  PUBLIC
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)

target_compile_features(kv_core PUBLIC cxx_std_20)

add_executable(kv_cli
  src/main.cpp
)

target_link_libraries(kv_cli PRIVATE kv_core)
```

4.2 PUBLIC, PRIVATE, INTERFACE

키워드	의미
PRIVATE	현재 타겟에서만 사용. 소비자에게 전파되지 않음
PUBLIC	현재 타겟도 사용하고 소비자에게도 전파
INTERFACE	현재 타겟 자체는 안 쓰고 소비자에게만 전파

전파 예시

```
target_include_directories(kv_core
  PUBLIC include
  PRIVATE src
)

target_link_libraries(kv_cli
  PRIVATE kv_core
)
```

위 예시에서 `kv_cli`는 `kv_core`를 링크하면서 `include` 경로는 자동으로 받지만, `src` 경로는 받지 않습니다.

Python 개발자 관점

이 개념은 Python 패키지의 공개 API와 내부 구현 경계를 나누는 것과 비슷합니다. `PUBLIC`은 라이브러리 사용자에게 노출되는 계약이고, `PRIVATE`은 내부 구현 세부사항입니다.

4.3 header-only 라이브러리

```
add_library(kv_flags INTERFACE)
target_compile_definitions(kv_flags INTERFACE KV_ENABLE_LOG=1)
target_include_directories(kv_flags INTERFACE include)
```

`INTERFACE` 라이브러리는 소스 파일이 없고, "사용 방법"만 배포합니다. 컴파일 옵션, include 디렉토리, 컴파일 정의를 묶는 데 유용합니다.

Quiz

Q3. `kv_core`의 헤더에서 `fmt::format`이 반환 타입과 인자 타입에 등장한다. `fmt`는 `PUBLIC`, `PRIVATE`, `INTERFACE` 중 무엇이어야 할까?
정답: 보통 `PUBLIC`. 헤더를 소비하는 쪽도 그 의존성을 알아야 하기 때문이다.

Chapter 5

프로젝트 구조를 어떻게 쪼개는가

5.1 권장 디렉토리 구조

```
project-root/
  CMakeLists.txt
  cmake/
    warnings.cmake
    sanitizers.cmake
  include/
    kv/store.h
  src/
    store.cpp
    main.cpp
  tests/
    store_test.cpp
  third_party/
```

5.2 루트와 서브디렉토리 책임 분리

루트 CMakeLists.txt

```cmake
cmake_minimum_required(VERSION 3.20)
project(kv_cli VERSION 0.1.0 LANGUAGES CXX)

option(KV_BUILD_TESTS "Build tests" ON)

add_subdirectory(src)

if(KV_BUILD_TESTS)
  enable_testing()
  add_subdirectory(tests)
endif()
```

src/CMakeLists.txt

```
add_library(kv_core
  store.cpp
)

target_include_directories(kv_core
  PUBLIC ${PROJECT_SOURCE_DIR}/include
)

add_executable(kv_cli main.cpp)
target_link_libraries(kv_cli PRIVATE kv_core)
```

주의

CMAKE_SOURCE_DIR와 PROJECT_SOURCE_DIR를 무분별하게 섞지 마십시오. 서브프로젝트로 포함될 가능성이 있으면 CMAKE_CURRENT_SOURCE_DIR, PROJECT_SOURCE_DIR, PROJECT_BINARY_DIR 처럼 문맥에 맞는 변수를 써야 합니다.

5.3 언제 add_subdirectory를 쓰나

- 우리 프로젝트의 일부인 내부 모듈: add_subdirectory
- 설치된 외부 패키지 찾기: find_package
- Git submodule이나 번들된 외부 라이브러리 직접 포함: 경우에 따라 add_subdirectory, 단 정책 충돌 주의

Quiz

Q4. 루트에서 include_directories()를 한 번 호출해서 모든 하위 타겟에 전역으로 뿌리는 방식이 왜 위험한가?
정답: 타겟 경계가 사라져서 어떤 헤더가 어디서 오는지 추적이 어려워지고, 외부 서브프로젝트와 충돌하기 쉽다. 타겟 단위 전파가 더 읽기 쉽고 재사용 가능하다.

Chapter 6

컴파일 옵션과 정의를 다루는 법

6.1 전역 옵션보다 타겟 옵션

권장 패턴

```
target_compile_features(kv_core PUBLIC cxx_std_20)

target_compile_options(kv_core PRIVATE
  -Wall
  -Wextra
  -Wpedantic
)

target_compile_definitions(kv_core PRIVATE
  KV_DATA_DIR="${CMAKE_CURRENT_SOURCE_DIR}/data"
)
```

6.2 빌드 타입과 generator expression

```
target_compile_definitions(kv_core PRIVATE
  $<$<CONFIG:Debug>:KV_ENABLE_ASSERTS=1>
  $<$<CONFIG:Release>:KV_FAST_PATH=1>
)
```

`$<...>`는 generator expression입니다. configure 시점이 아니라 generate/build 시점에 해석됩니다.

Multi-config vs Single-config

- Ninja, Makefiles: 보통 single-config. `CMAKE_BUILD_TYPE` 사용.
- Visual Studio, Xcode, Ninja Multi-Config: 여러 구성을 동시에 가짐.
- 그래서 실전에서는 `if(CMAKE_BUILD_TYPE STREQUAL ...)`보다 generator expression이

더 이식성 있다.

6.3 경고 플래그 모듈화

cmake/warnings.cmake

```
add_library(project_warnings INTERFACE)

target_compile_options(project_warnings INTERFACE
  $<$<CXX_COMPILER_ID:Clang,AppleClang,GNU>:-Wall;-Wextra;-Wpedantic>
  $<$<CXX_COMPILER_ID:MSVC>:/W4>
)
```

```
target_link_libraries(kv_core PRIVATE project_warnings)
```

Quiz

Q5. `add_compile_options(-Wall)` 와 `target_compile_options(kv_core PRIVATE -Wall)` 중 후자가 더 나은 이유는?

정답: 어떤 타겟이 어떤 규칙을 가지는지 국소적으로 드러나고, 외부 서브프로젝트나 특정 타겟에 원치 않는 옵션 전파를 막을 수 있기 때문이다.

Chapter 7

링크, 라이브러리, 그리고 왜 자꾸 undefined reference가 나는가

7.1 컴파일과 링크는 다른 단계다

- 컴파일: 각 `.cpp`를 객체 파일로 변환
- 링크: 객체 파일과 라이브러리를 합쳐 최종 실행 파일 생성
- `undefined reference`: 헤더는 보이는데 구현 심볼이 링크에 안 들어왔다는 뜻

7.2 정적 라이브러리와 공유 라이브러리

```
add_library(kv_core STATIC
  src/store.cpp
)

add_library(kv_plugin SHARED
  src/plugin.cpp
)
```

실전 감각

- 앱 내부 모듈은 대부분 `STATIC` 또는 기본 `add_library`로 충분하다.
- 플러그인 시스템, ABI 노출, 동적 교체가 필요할 때만 `SHARED`를 신중하게 검토한다.
- 초반에는 "링크 그래프를 명확하게" 만드는 것이 동적 로딩보다 더 중요하다.

7.3 의존성 전파 예시

```
add_library(kv_storage src/store.cpp)
add_library(kv_commands src/command.cpp)

target_link_libraries(kv_commands PUBLIC kv_storage)
```

```
5
6 add_executable(kv_cli src/main.cpp)
7 target_link_libraries(kv_cli PRIVATE kv_commands)
```

위 경우 `kv_cli`는 `kv_storage`를 직접 쓰지 않아도, `kv_commands`의 `PUBLIC` 링크 전파 덕분에 함께 링크됩니다.

주의

링크가 우연히 통과한다고 해서 설계가 좋은 것은 아닙니다. 헤더에 노출된 의존성은 `PUBLIC`, 구현에만 필요한 의존성은 `PRIVATE`로 정리해야 나중에 리팩터링이 쉬워집니다.

Quiz

Q6. `undefined reference to KVStore::set(...)` 에러가 났다. 가능한 원인을 두 가지 써 보라.

정답 예시: `store.cpp`가 타겟 소스 목록에서 빠졌거나, `kv_core` 라이브러리를 최종 실행 파일에 링크하지 않았다.

Chapter 8

외부 패키지를 찾고 연결하기

8.1 find_package의 두 세계

1. **Module mode**: CMake가 제공하는 `FindXXX.cmake`를 찾는다.
2. **Config mode**: 패키지가 설치한 `XXXConfig.cmake`를 찾는다.

```
find_package(fmt REQUIRED CONFIG)
find_package(spdlog REQUIRED CONFIG)
```

현대 CMake에서는 가능하면 Config mode와 imported target을 쓰는 것이 좋습니다.

8.2 Imported target을 써야 하는 이유

```
target_link_libraries(kv_core
  PUBLIC
    fmt::fmt
    spdlog::spdlog
)
```

이 방식은 include path, compile definition, 필요한 링크 플래그를 패키지 작성자가 한 번 묶어서 제공합니다.

주의

`include_directories(/usr/local/include)`와 `link_directories(/usr/local/lib)`를 직접 쓰는 방식은 현대 CMake에서 거의 항상 열세다. 패키지가 제공하는 imported target이 있으면 그걸 우선 사용하라.

8.3 패키지를 못 찾을 때 보는 것

- `CMAKE_PREFIX_PATH`
- 패키지별 `<Pkg>_DIR`
- 설치 위치 안의 `lib/cmake/<Pkg>` 또는 유사 경로

- 아키텍처 불일치 여부

예시

```
cmake -S . -B build \
  -DCMAKE_PREFIX_PATH=/opt/homebrew \
  -Dfmt_DIR=/opt/homebrew/lib/cmake/fmt
```

Quiz

Q7. `find_package(MLIR REQUIRED CONFIG)`가 실패했다. 가장 먼저 확인할 것은?
정답: MLIR 설치 경로 아래의 `MLIRConfig.cmake` 위치와, 그 위치를 가리키는 `MLIR_DIR` 또는 `CMAKE_PREFIX_PATH` 설정.

Chapter 9

테스트, install, package로 이어지는 흐름

9.1 CTest 연결

```
include(CTest)

if(BUILD_TESTING)
  add_executable(store_test tests/store_test.cpp)
  target_link_libraries(store_test PRIVATE kv_core)
  add_test(NAME store_test COMMAND store_test)
endif()
```

```
ctest --test-dir build --output-on-failure
```

9.2 install 규칙

```
install(TARGETS kv_core kv_cli
  EXPORT kvTargets
  RUNTIME DESTINATION bin
  LIBRARY DESTINATION lib
  ARCHIVE DESTINATION lib
)

install(DIRECTORY include/ DESTINATION include)
```

9.3 재사용 가능한 패키지로 내보내기

```
install(EXPORT kvTargets
  NAMESPACE kv::
  DESTINATION lib/cmake/kv
```

```
)
```

이 단계까지 가면 다른 프로젝트에서 `find_package(kv CONFIG REQUIRED)` 후 `kv::kv_core` 같은 imported target으로 소비할 수 있습니다.

Python 개발자 관점

이 감각은 Python 패키지를 wheel로 배포해서 다른 프로젝트가 import 하는 흐름과 닮았습니다. 차이는 C++은 ABI, 헤더, 링크 옵션까지 함께 전달해야 한다는 점입니다.

Quiz

Q8. 왜 `install()` 없이도 로컬 빌드는 되지만, 다른 프로젝트에서 재사용하기는 어려울까?
정답: 설치 규칙이 없으면 헤더, 라이브러리, export target 메타데이터를 일관된 위치와 형식으로 제공하지 못해 외부 프로젝트가 안정적으로 소비할 수 없다.

Chapter 10

configure 단계에서 파일을 생성하는 법

10.1 configure_file

```
configure_file(
  ${CMAKE_CURRENT_SOURCE_DIR}/config.h.in
  ${CMAKE_CURRENT_BINARY_DIR}/generated/config.h
  @ONLY
)
```

config.h.in

```
#pragma once

#define KV_VERSION "@PROJECT_VERSION@"
#define KV_ENABLE_CACHE @KV_ENABLE_CACHE@
```

`configure_file()`는 configure 시점 치환입니다. 빌드할 때마다 실행되는 스크립트가 아니라, 설정이 바뀔 때 다시 생성되는 파일입니다.

10.2 file(GENERATE)와의 차이

- `configure_file`: configure 시점 치환
- `file(GENERATE)`: generate 시점, configuration별 파일 생성에 유리
- `add_custom_command`: build 시점에 실제 작업 수행

주의

configure 단계에서 해야 할 일과 build 단계에서 해야 할 일을 혼동하면, 소스를 바꿨는데 파일이 안 갱신되거나 매번 불필요하게 재생성되는 문제가 생긴다.

Quiz

Q9. protobuf 컴파일, TableGen 생성, codegen 스크립트 실행은 보통 `configure_file()`보다 무엇에 가깝나?

정답: `add_custom_command()` 혹은 해당 생태계가 제공하는 전용 매크로. 실제 빌드 산출물 생성 단계이기 때문이다.

Chapter 11

Presets와 Toolchain으로 팀 개발 수준을 올리기

11.1 CMakePresets.json

개념 예시, 실제 파일은 JSON

```
{
  "version": 6,
  "configurePresets": [
    {
      "name": "dev",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build/dev",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "KV_BUILD_TESTS": "ON"
      }
    }
  ]
}
```

Presets는 팀 전체가 같은 configure 인자를 재현하도록 도와줍니다. README에 긴 명령을 적는 대신, `cmake --preset dev`로 고정할 수 있습니다.

11.2 Toolchain 파일

```
set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_C_COMPILER clang)
set(CMAKE_CXX_COMPILER clang++)
```

Toolchain 파일은 cross-compiling, 특정 SDK, sysroot, 패키지 매니저 통합에 유용합니다.

실전 체크리스트

- 팀에 빌드 명령이 길어지면 Presets를 도입한다.
- 플랫폼이나 아키텍처 차이를 숨겨야 하면 Toolchain 파일을 고려한다.
- 문서보다 재현 가능한 설정 파일을 우선한다.

Quiz

Q10. 개인 환경별로 `CMAKE_PREFIX_PATH`가 다르고, 팀원이 매번 README의 긴 명령을 복붙하고 있다. 무엇을 도입하면 좋은가?

정답: 기본적으로 `CMakePresets.json`. 플랫폼/컴파일러 자체까지 고정해야 하면 필요에 따라 toolchain 파일도 함께 사용.

Chapter 12

LLVM/MLIR 프로젝트를 읽는 눈

12.1 왜 별도 장이 필요한가

당신은 단순한 toy project보다 한 단계 더 가려 하고 있습니다. LLVM/MLIR 기반 프로젝트는 CMake 파일이 길고 매크로가 많으며, 링크 의존성이 빽빽합니다. 여기서 중요한 것은 문법 암기보다 **패턴 인식**입니다.

12.2 핵심 패턴

전형적인 MLIR 설정

```cmake
find_package(MLIR REQUIRED CONFIG)
find_package(LLVM REQUIRED CONFIG)

list(APPEND CMAKE_MODULE_PATH "${MLIR_CMAKE_DIR}")
list(APPEND CMAKE_MODULE_PATH "${LLVM_CMAKE_DIR}")

include(TableGen)
include(AddLLVM)
include(AddMLIR)
```

```cmake
if(NOT LLVM_ENABLE_RTTI)
  add_compile_options(-fno-rtti)
endif()
```

여기서 읽어야 할 포인트

1. 패키지 설정을 어디서 찾는가
2. 생태계 전용 매크로를 어떻게 로드하는가
3. 프로젝트 컴파일 옵션이 상위 생태계와 어떤 제약을 공유하는가
4. 어떤 라이브러리가 어떤 dialect/pass/transform 계층을 담당하는가

12.3 라이브러리 계층을 읽는 법

```
add_library(GaweeDialect
  lib/Gawee/GaweeDialect.cpp
)

target_link_libraries(GaweeDialect
  PUBLIC
    MLIRIR
    MLIRSupport
)

add_library(GaweeConversion
  lib/Conversion/GaweeToLinalg.cpp
)

target_link_libraries(GaweeConversion
  PUBLIC
    GaweeDialect
  PRIVATE
    MLIRLinalgDialect
    MLIRTransforms
)
```

여기서 `GaweeConversion`의 공개 API가 `GaweeDialect` 타입을 노출한다면 `PUBLIC`이 맞고, 내부 lowering 구현만 `MLIRLinalgDialect`를 쓰면 `PRIVATE`가 맞습니다.

12.4 빈출 문제

증상	보통 원인
`typeinfo for mlir::Pass` 관련 링크 에러	RTTI 설정 불일치
`Could not find MLIRConfig.cmake`	`MLIR_DIR` 또는 prefix 경로 미설정
`file not found` 헤더 에러	include 경로 또는 generated header 경로 누락
`undefined reference` 패스 생성 함수	관련 라이브러리 미링크

Quiz

Q11. LLVM이 RTTI 없이 빌드되었는데 우리 프로젝트는 RTTI를 켠 상태다. 가장 가능성 높은 결과는?

정답: 컴파일은 되더라도 링크 단계에서 `typeinfo` 관련 에러가 날 수 있다. 상위 생태계와 설정을 맞춰야 한다.

Chapter 13

디버깅과 안티패턴

13.1 문제가 생겼을 때 순서

1. configure 에러인가, build 에러인가, test/install 에러인가 구분한다.
2. 문제가 난 타겟이 무엇인지 본다.
3. 그 타겟의 소스, include path, compile definition, link dependency를 확인한다.
4. 전역 변수와 매크로가 흐름을 숨기고 있지 않은지 본다.

13.2 유용한 명령

```
cmake -S . -B build --trace-expand
cmake --build build --verbose
cmake --graphviz=graph.dot -S . -B build
ctest --test-dir build --output-on-failure
```

13.3 피해야 할 안티패턴

- 루트에서 `include_directories()`, `link_libraries()`를 전역 살포
- 패키지 대신 절대 경로 하드코딩
- `file(GLOB ...)`로 소스를 자동 수집해서 변경 추적 불투명하게 만들기
- 옵션과 매크로를 함수처럼 섞어 타겟 경계를 흐리기
- "일단 돌아가니까" PUBLIC/PRIVATE를 대충 두기

주의

`file(GLOB)`는 데모에는 편하지만, 팀 프로젝트에서는 새 파일 추가가 configure 재실행과 엮여 예측 가능성을 떨어뜨린다. 명시적 소스 목록이 더 유지보수에 강하다.

Quiz

Q12. CMake 디버깅에서 가장 먼저 해야 할 질문은 "어느 파일에서 문제가 났나?"가 아니라 무엇인가?
정답: "어느 타겟의 어떤 속성에서 문제가 났나?" 파일보다 타겟이 빌드 그래프의 기본 단위이기 때문이다.

Chapter 14

당장 실전에 적용할 로드맵

14.1 1단계: 작은 CLI 프로젝트

- `add_executable` 하나로 시작
- `build/` 분리
- `target_compile_features`, `target_include_directories`, `target_link_libraries`를 익힌다

14.2 2단계: 라이브러리 분리

- `kv_core`, `kv_cli`로 분리
- PUBLIC/PRIVATE를 의식적으로 설정
- 테스트 타겟 추가

14.3 3단계: 외부 패키지와 설치

- `find_package`와 imported target 사용
- `install()`과 export target 구성
- Presets로 팀 환경 고정

14.4 4단계: 대형 생태계 읽기

- LLVM/MLIR CMake를 읽을 때 타겟 그래프를 먼저 본다
- 생태계 전용 매크로가 실제로 어떤 타겟을 만드는지 추적한다
- 에러를 보면 "헤더 경로", "심볼 정의", "상위 설정 불일치" 중 무엇인지 분류한다

배운 것 정리

당신의 실제 목표는 "CMake 문법을 많이 아는 사람"이 아니라, **복잡한 C++ 프로젝트를 겁먹지 않고 구조화하고 디버깅하는 사람**이 되는 것입니다. mini-kvstore 같은 작은 CLI에서 시작해, 라이브러리 분리, 테스트, 패키지, LLVM/MLIR 연동까지 자연스럽게 올라가면 됩니다.

Chapter 15

Final Quiz

종합 문제 1

다음 상황을 읽고 올바른 설계를 설명하라.

- `kv_core` 라이브러리가 공개 헤더에서 `fmt::format` 결과를 반환한다.
- `kv_cli` 실행 파일이 `kv_core`를 사용한다.
- 테스트 타겟 `store_test`도 `kv_core`를 사용한다.

해설: `kv_core`는 `fmt::fmt`를 `PUBLIC`으로 링크해야 할 가능성이 높다. `kv_cli`와 `store_test`는 `kv_core`만 링크하면 전파받을 수 있다.

종합 문제 2

다음 중 대형 프로젝트에서 가장 먼저 리팩터링해야 할 냄새는?

A. 타겟별 `target_link_libraries` 사용

B. 전역 `include_directories()` 남발

C. `find_package(fmt CONFIG REQUIRED)` 사용

D. `install(EXPORT ...)` 제공

정답: B

종합 문제 3

`gawee-opt` 링크 중 `undefined reference to mlir::linalg::GenericOp::create`가 난다. 가장 그럴듯한 원인은?

정답: 관련 dialect/transform 라이브러리가 `target_link_libraries`에 빠졌을 가능성이 높다. 헤더 include만으로는 링크 심볼이 해결되지 않는다.

Appendix A

암기보다 중요한 체크포인트

실전 체크리스트

1. 모든 것은 타겟 중심으로 읽고 쓴다.
2. 전역 설정보다 타겟 속성을 우선한다.
3. 패키지는 경로 하드코딩보다 imported target을 우선한다.
4. configure/build/install/test의 단계를 섞지 않는다.
5. 대형 프로젝트에서는 상위 생태계의 제약을 먼저 확인한다.

Appendix B

빠른 명령 치트시트

```
# Configure
cmake -S . -B build/dev -G Ninja -DCMAKE_BUILD_TYPE=Debug

# Build
cmake --build build/dev

# Run tests
ctest --test-dir build/dev --output-on-failure

# Install
cmake --install build/dev --prefix ./stage

# Trace and debug
cmake -S . -B build/dev --trace-expand
cmake --build build/dev --verbose
```